Lautaro Agustín Montoya

Game Designer — Sistemas y mecánicas · Unity / C#

Córdoba, Argentina (disponible para trabajo remoto) · +54 9 11 3557-3316 · lautaroamontoya@gmail.com · LinkedIn

Portfolio: montoyalautaroportfolio.netlify.app · Jugá Keep It Alive: pehuamann.itch.io/keep-it-alive

PERFIL

Game designer enfocado en sistemas, que diseña y construye sus propios juegos de punta a punta. Actualmente desarrollo **Keep It Alive**, un roguelite deckbuilder por turnos sobre proteger a un ser querido internado, hoy en acceso anticipado público en itch.io, junto con otros cinco prototipos y conceptos originales de mi portfolio. Prototipo en Unity y C#, estructuro el contenido como datos y ajusto el balance con simulaciones automáticas y pruebas de juego. Cinco años en viajes y atención al cliente me enseñaron a decidir bajo presión y a comunicar con claridad entre equipos. Busco un puesto de **Junior Game Designer** o **Systems Designer**.

PROYECTO DESTACADO

Keep It Alive — Diseñador y desarrollador (proyecto individual) Acceso anticipado en itch.io

Diseño y dirección · Unity, C#, ScriptableObjects

Roguelite deckbuilder por turnos, para un jugador, en pixel art. El jugador protege a un ser querido internado, su Protegido, en batallas de cartas día a día contra las dificultades de una internación, hasta quedar cara a cara con La Muerte. Objetivo de diseño: convertir el agotamiento en un sistema que cambie las decisiones, no en una penalización plana.

- **Enemigos que atacan la mano del jugador:** la Fiebre quema cartas, la Depresión las moja y las inutiliza, y otras amenazas las atrapan, las mutan o las borran; cada enemigo cambia lo que el jugador puede hacer, no solo cuánto daño recibe.
- **Cadena de recursos que escala:** 3 de Energía y 5 cartas por turno; la Sobreexigencia permite jugar cartas sin Energía suficiente y paga lo que falta con el doble de Fatiga. La Fatiga (0–10) suma cartas de estado al mazo en cada nivel, termina en Colapso (una carta por turno) y se arrastra entre peleas.
- **Dos vidas, dos tipos de protección:** la vida del jugador y la del Protegido se conservan entre encuentros y la partida termina si cualquiera llega a 0; la Estabilización protege al Protegido de forma persistente y la Contención protege al jugador solo por un momento.
- **Estructura de la partida:** un camino con ramas y cuatro tipos de nodo (días comunes, días más difíciles con mejores recompensas, tiendas donde las cartas se pagan con la vida del jugador y noches de descanso), 8 enemigos comunes, 5 jefes que transforman el cuarto y un encuentro final con La Muerte que no se gana haciendo daño.
- **Contenido a escala:** 108 cartas del jugador y 52 cartas enemigas y de estado, todas definidas en tablas de datos, para diseñar y rebalancear contenido sin tocar código.
- **Balance por simulación:** construí un motor de reglas sin interfaz donde bots con distintos estilos juegan miles de partidas completas (5.000 por ajuste) y reportan victorias contra un objetivo de 20–35%, jefes superados y causas de muerte; 122 pruebas automáticas protegen las reglas.
- **Iteración a partir de feedback:** una prueba mostró que un enemigo común pegaba más que un jefe; la simulación confirmó que una carta (*Código Azul*) era la causa de muerte n.º 1 (13,7%). La saqué del mazo de ese enemigo y fijé la regla "un enemigo común nunca puede ser más peligroso que un jefe"; las victorias pasaron de 27% a 36%.
- **Pensado para rejugar:** tres niveles de dificultad (medidos en 67% / 40% / 17% de victorias), cartas que se desbloquean al llegar a La Muerte y soporte completo de mouse, teclado y joystick.
- Diseño e implementación de punta a punta en Unity y C#: sistemas de juego, datos de cartas, interfaz con prefabs editables, mecánicas de enemigos y cada decisión registrada en un documento de diseño.

OTROS PROYECTOS

Junkborn — Eco de Chatarra Prototipo jugable · Unity

Prototipo de sistemas roguelite. El jugador arma un robot con 5 espacios de componentes que se degradan en combate, intercambiando poder por durabilidad. Hecho para comprobar si la degradación del equipo genera improvisación en lugar de castigo; el prototipo registra turnos sobrevividos, enemigos derrotados y componentes rotos.

Deuda de Sangre (Blood Debt) Juego de cartas estratégico · En desarrollo

El jugador juzga almas como cobrador de un puente espiritual; sistemas como los Puntos de Carga y el Anclaje Terrenal conectan la historia de cada alma con consecuencias de juego y corrupción.

Conceptos de diseño Casos de estudio en el portfolio

Grey Market (simulación sistémica de una economía impulsada por la propagación de información), Desfase — Shift Protocol (puzzle 3D sobre estados inestables de la realidad) y Último Aliento (supervivencia por turnos donde el oxígeno es la moneda táctica).

HABILIDADES

Diseño de juegos	Diseño de sistemas, loops centrales, diseño de recursos y economía, diseño de cartas y deckbuilding, estructura de partida y progresión, diseño de jefes y encuentros, balance, mecánicas de riesgo y recompensa, sistemas de estado del jugador, prototipado rápido, pruebas de juego e iteración, documentación de diseño.
Técnicas	Unity, C#, diseño guiado por datos (tablas de datos, ScriptableObjects), simulación y pruebas automáticas de balance, Git / GitHub.
Profesionales	Comunicación escrita clara, coordinación con distintas partes, toma de decisiones bajo presión, organización de muchas tareas en paralelo.

EXPERIENCIA

Coordinador de viajes — Newfly Córdoba, Argentina · 2024 – actualidad

- Coordino la logística de viajeros individuales y grupos, con muchos casos en simultáneo junto a aerolíneas, hoteles y operadores locales.
- Resuelvo cancelaciones y cambios de último momento con rapidez, eligiendo entre opciones con información incompleta y plazos ajustados.
- Mantengo a todas las partes alineadas con actualizaciones escritas claras, para que clientes, proveedores y compañeros sigan el mismo plan.

Atención al cliente — KMB, Eilat, Distraction Good Tourism, Konecta 2021 – 2023

- Atención a clientes en español, inglés y portugués en turismo y centros de contacto.

FORMACIÓN

Desarrollo de videojuegos autodidacta 1–2 años

Unity, programación de gameplay en C#, arquitectura guiada por datos, pruebas y simulación automáticas, flujo de trabajo con Git; aplicado directamente en Keep It Alive.

IDIOMAS

Español (nativo) · Inglés (fluido) · Portugués (fluido) · Alemán (en aprendizaje)